# LABORATORIO NRF_24LO1

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen - En esta práctica se implementó un sistema de comunicación digital inalámbrica con dos Raspberry Pi Pico 2W y módulos transceptores nRF24L01+, y se analizó la interfaz SPI entre el microcontrolador y el módulo mediante un analizador lógico (Logic 2) conectado a las líneas SCK, MOSI, MISO y CSN. Se capturaron y decodificaron tres operaciones: la lectura del registro SETUP_AW, la escritura del registro RF_CH que fija el canal de radiofrecuencia (canal 96, cuya frecuencia calculada es 2496 MHz) y la carga de un paquete con el comando W_TX_PAYLOAD. De cada una se identificaron el comando, los bytes transmitidos por MOSI y MISO, el comportamiento de CSN y los tiempos de transferencia.

Palabras clave - analizador lógico, GFSK, MicroPython, nRF24L01+, Raspberry Pi Pico 2W, SPI.

Abstract - This practical exercise implemented a digital wireless communication system using two Raspberry Pi Pico 2W boards and nRF24L01+ transceivers, and analyzed the SPI interface between the microcontroller and the module using a logic analyzer (Logic 2) connected to the SCK, MOSI, MISO and CSN lines. Three operations were captured and decoded: a read of the SETUP_AW register, a write to the RF_CH register that sets the radio channel (channel 96, with a calculated frequency of 2496 MHz), and a packet load with the W_TX_PAYLOAD command. For each one, the command, the bytes sent over MOSI and MISO, the CSN behavior and the transfer times were identified.

Keywords - GFSK, logic analyzer, MicroPython, nRF24L01+, Raspberry Pi Pico 2W, SPI.

## I. INTRODUCCIÓN

Los sistemas de comunicación digital inalámbrica permiten transmitir información sin una conexión física entre dispositivos. Para configurar y controlar módulos de radio como el nRF24L01+, los microcontroladores utilizan interfaces de comunicación como SPI (Serial Peripheral Interface), mediante la cual se intercambian comandos y datos.

En esta práctica se implementó un sistema de comunicación inalámbrica con dos placas Raspberry Pi Pico 2W y módulos transceptores nRF24L01+. De acuerdo con las indicaciones del docente, el análisis se delimitó a la prueba 4.1 de la guía, correspondiente a la interfaz SPI. Se capturaron operaciones de lectura de registros, escritura del canal de radiofrecuencia y carga de un paquete en el transceptor.

A partir de las capturas se identificaron los comandos, los bytes transmitidos por MOSI y recibidos por MISO, el comportamiento de CSN y los tiempos de transferencia. Estos datos permitieron comparar las mediciones con el modelo teórico y reconocer las diferencias asociadas a las pausas entre bytes y al control de la interfaz por software.

## II. OBJETIVOS

### Objetivo General

Analizar la capa física y la estructura de las transacciones del bus SPI entre una Raspberry Pi Pico 2W y el transceptor nRF24L01+, relacionando las señales capturadas con el comportamiento teórico del protocolo.

### Objetivos específicos

- Interpretar la respuesta del módulo por MISO (registro STATUS y contenido del registro leído) durante las transacciones SPI capturadas.
- Identificar el comando, los bytes de MOSI y MISO y el comportamiento de CSN en una lectura de registro, en una escritura del canal RF_CH y en la carga de un paquete.
- Calcular la frecuencia de portadora a partir del valor escrito en RF_CH.
- Comparar los tiempos de transferencia medidos con el tiempo teórico t = 8n/f_SCK, calcular el error relativo y explicar sus causas.

## III. MARCO TEÓRICO

### A. Comunicación inalámbrica en la banda ISM de 2.4 GHz

La banda ISM de 2,4 GHz comprende aproximadamente de 2400 a 2483,5 MHz y es compartida por tecnologías como Wi-Fi, Bluetooth y otros dispositivos de corto alcance. Al ser una banda compartida, los sistemas que operan en ella pueden experimentar interferencias [1]. La frecuencia configurada debe mantenerse dentro de los límites aplicables al servicio y al lugar de operación.

### B. Modulación GFSK

Es una técnica de modulación digital que representa la información mediante cambios en la frecuencia de una portadora. Se diferencia de la FSK convencional porque utiliza un filtro gaussiano antes de la modulación, suavizando las transiciones de frecuencia y reduciendo el ancho de banda ocupado. El nRF24L01+ emplea esta técnica para transmitir datos de forma inalámbrica en la banda de 2,4 GHz [1].

### A. C. Transceptor nRF24L01+

El nRF24L01+ es un transceptor de radiofrecuencia para la transmisión y recepción de datos. Permite configurar parámetros como el canal, la velocidad de transmisión y la potencia de salida. El registro RF_CH define la frecuencia central mediante la relación indicada en la ecuación (3).

El canal de operación se establece mediante el registro RF_CH, cuyo valor determina la frecuencia central de transmisión según la expresión:

$$f = 2400\,MHz + RF_{CH}\ \ MHz \qquad (1)$$

El valor de RF_CH es un entero entre 0 y 125; según la relación del fabricante.

### D. Interfaz de comunicación SPI

SPI es un protocolo serial síncrono y dúplex completo que permite intercambiar información entre un maestro y un esclavo. En esta práctica, la Raspberry Pi Pico 2W actúa como maestro y el nRF24L01+ como esclavo. La interfaz utiliza cuatro señales principales:

- SCK: señal de reloj generada por el maestro.
- MOSI: transmite datos del maestro al esclavo.
- MISO: transmite datos del esclavo al maestro.
- CSN: selecciona el dispositivo y permanece en nivel bajo durante la transacción.

El nRF24L01+ trabaja en modo SPI 0 (CPOL = 0 y CPHA = 0), transmite primero el bit más significativo (MSB) y organiza los datos en bytes de ocho bits [1].

El tiempo ideal necesario para transferir n bytes, cada uno de ocho bits, a una frecuencia de reloj f_SCK se calcula mediante la siguiente ecuación

$$t_{teo} = \frac{(8 \times n)}{f_{sck}} \qquad (2)$$

Para comparar con la medición se usa el error relativo:

$$E\% = \frac{|t_{med} - t_{teo}|}{t_{teo}} \qquad (3)$$

### E. Comandos SPI del nRF24L01+

Cada transacción SPI comienza con un byte de comando. Mientras el maestro lo transmite por MOSI, el nRF24L01+ devuelve por MISO el registro STATUS. Después del reinicio, el valor observado para STATUS es 0x0E [1]. Los comandos y registros relevantes se resumen en la Tabla I.

| Comando / registro | Valor | Función |
|---|---|---|
| R_REGISTER | 0x00 \| dirección | Lee un registro |
| W_REGISTER | 0x20 \| dirección | Escribe un registro |
| W_TX_PAYLOAD | 0xA0 | Carga el paquete a transmitir |
| NOP | 0xFF | Sin operación; genera reloj |
| SETUP_AW | dirección 0x03 | Ancho de dirección |
| RF_CH | dirección 0x05 | Canal de RF |
| STATUS | dirección 0x07 | Estado e interrupciones |

**Tabla 1.Comandos y registros del nRF24L01+.**

### F. Raspberry Pi Pico

La Raspberry Pi Pico 2W utiliza el microcontrolador RP2350 e incorpora interfaces SPI por hardware y GPIO de 3,3 V, compatibles eléctricamente con el nRF24L01+ [3].

### G. Analizador lógico

El analizador lógico registra el nivel digital de varias líneas en el tiempo. Para capturar una señal digital sin perder flancos, la frecuencia de muestreo debe ser varias veces mayor que la frecuencia de la señal. El software asociado puede además decodificar el protocolo y mostrar los bytes transmitidos por MOSI y MISO, junto con el tiempo de cada transacción, lo que permite verificar comandos, datos y señales de control del bus [4].

## IV. METODOLOGÍA

### A. Construcción del sistema

Se implementó un sistema de comunicación inalámbrica compuesto por dos nodos: un transmisor (TX) y un receptor (RX). Cada nodo utiliza una Raspberry Pi Pico 2W, un transceptor nRF24L01+, una pantalla OLED SSD1306 conectada por I2C, dos pulsadores y tres LED con sus respectivas resistencias limitadoras de corriente.

Los pulsadores se utilizaron para recorrer y confirmar las opciones de configuración del sistema, mientras que los LED permitieron identificar los estados de funcionamiento. La pantalla OLED se empleó para visualizar los parámetros configurados y los mensajes correspondientes a cada nodo.

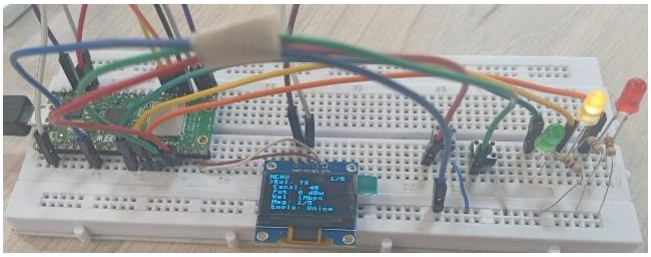


**Figure 1 Montaje experimental del transmisor (TX).**

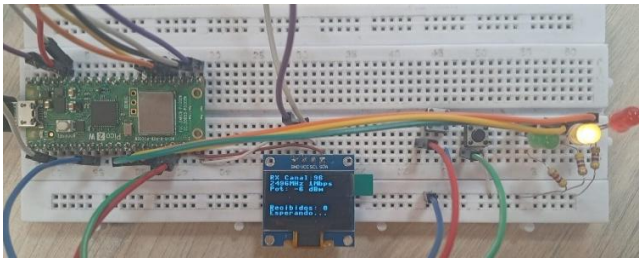


**Figure 2.Montaje experimental del receptor (RX).**

El análisis experimental se centró en la interfaz SPI entre la Raspberry Pi Pico 2W del transmisor y el nRF24L01+. Se observaron una lectura de registro, una escritura del registro RF_CH y la carga de un mensaje en el transceptor. Estas pruebas permitieron identificar comandos, respuestas y temporización del bus.

### B. Conexiones Implementadas

Se conectaron los periféricos a la Raspberry Pi Pico 2W: la pantalla OLED SSD1306, con dirección I2C 0x3C; el transceptor nRF24L01+, mediante SPI; y el analizador lógico, conectado a las señales SPI del nodo transmisor.

Las conexiones son iguales en ambas placas:

| Elemento | Señal | Pin Pico |
|---|---|---|
| nRF24L01 | SCK | GP18 |
| nRF24L01 | MOSI | GP19 |
| nRF24L01 | MISO | GP16 |
| nRF24L01 | CSN | GP17 |
| nRF24L01 | CE | GP20 |
| OLED | SDA / SCL | GP14 / GP15 |
| Pulsador variar | entrada | GP13 |
| Pulsador seleccionar | entrada | GP12 |
| LED rojo (error) | salida | GP11 |
| LED amarillo (configurando) | salida | GP10 |
| LED verde (enviado/recibido) | salida | GP9 |

**Tabla 2. Conexiones Raspberry y periféricos**.

### C. Diseño del software

El firmware, escrito en MicroPython, se divide en dos archivos:

nrf24.py: controlador del módulo. Encapsula las operaciones SPI (lectura y escritura de registros, carga de payload, selección de canal, velocidad y potencia).

main.py: lógica de la aplicación.

El funcionamiento del sistema es el siguiente:

- Arranque: se muestra un mensaje de bienvenida en la OLED, y luego el canal, la velocidad de transmisión y la potencia configurados.
- Configuración: el pulsador «variar» permite recorrer las opciones y «seleccionar» confirma la elección. Se pueden configurar canal, potencia, velocidad y mensaje. Aunque el firmware permite seleccionar los canales disponibles del módulo, los canales por encima de 2483,5 MHz quedan fuera de la banda ISM de 2,4 GHz.
- Mensajes: el TX dispone de una lista con mensajes cortos (nombres de los estudiantes) y largos (alfabetos), además de un modo de envío en bucle.
- Indicación con LEDs: el rojo señala un error, el amarillo indica que el sistema está configurando o esperando, y el verde indica que un mensaje fue enviado o recibido.
- El RX permanece escuchando en el mismo canal y velocidad, y muestra en la OLED el mensaje recibido.

El código del firmware se incluye en el **Anexo A**

### C. Configuración del analizador lógico

Para observar la comunicación entre la Raspberry Pi Pico 2W y el nRF24L01+ se conectó un analizador lógico al nodo TX y se utilizó el software Logic 2 [4]. Se emplearon cuatro canales, conectados de la siguiente manera:

Channel 1: SCK (GP18)
Channel 2: MOSI (GP19)
Channel 3: MISO (GP16)
Channel 4: CSN (GP17)

En Logic 2 se habilitó el decodificador SPI y se asignó el canal 4 a CSN, utilizando su flanco de bajada para identificar el inicio de cada transacción. La frecuencia de muestreo se configuró en 12 MS/s, suficiente para observar el reloj SPI de 1 MHz y decodificar los bytes capturados.

## V. PRUEBAS Y ANÁLISIS DE RESULTADOS

### A. Análisis de SPI

Para cada operación se inició la captura y se ejecutó en el TX la acción correspondiente: reinicio de la placa para observar la lectura de registro, cambio de canal mediante los pulsadores para provocar la escritura y envío del mensaje para capturar la carga del paquete. En Logic 2 se identificaron el comando, los bytes de MOSI y MISO, el comportamiento de CSN y los

tiempos registrados. El tiempo teórico se calculó con la ecuación (4) y el error relativo con la ecuación (5).

### B. Lectura de registros

Para observar una lectura, se inició la captura del analizador lógico antes de reiniciar el nodo transmisor. Durante el arranque, la Raspberry Pi Pico 2W ejecuta operaciones de configuración y lectura de registros del nRF24L01+. En la captura se seleccionó la transacción correspondiente al registro SETUP_AW.

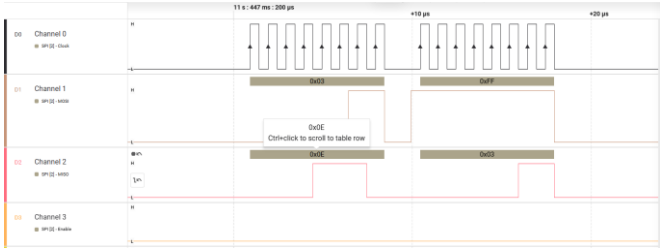



**Figure 3. Conexión del analizador lógico al nodo TX.**

En la captura se identificó la transacción de lectura y se verificaron las señales CSN, SCK, MOSI y MISO. CSN permanece en nivel bajo durante la transacción y vuelve a nivel alto al finalizarla.

La lectura corresponde al comando R_REGISTER para la dirección del registro SETUP_AW (0x03). El comando se obtiene aplicando la operación OR entre 0x00 y la dirección 0x03:

R_REGISTER = 0x00 OR 0x03 = 0x03

Durante el primer byte, la Pico transmite el comando 0x03 por MOSI y el nRF24L01+ devuelve el byte STATUS por MISO (0x0E en la captura). Para recibir el contenido del registro, el maestro transmite después 0xFF (NOP), que genera los pulsos de reloj; el módulo devuelve 0x03, correspondiente al ancho de dirección configurado de cinco bytes.

Para dos bytes y una frecuencia de reloj f_SCK = 1 MHz, el tiempo ideal calculado mediante la ecuación (2) es:

$$t_{teo} = \frac{8 \times 2}{1\,MHz} = 16\mu s$$

Con el tiempo de reloj activo medido de 17 µs, el error relativo respecto al valor teórico se calcula mediante la ecuación (3):

$$E\% = \frac{|17 - 16|}{16} \times 100 = 6.25\ \%$$

El tiempo durante el cual CSN permanece en bajo es de 101,75 µs, aproximadamente 5,99 veces el tiempo de reloj activo (101,75/17 = 5,99). La diferencia de 84,75 µs entre ambos valores corresponde al tiempo adicional observado entre la activación y desactivación de CSN. Esta duración incluye la gestión de la transacción por el firmware y no debe confundirse con el tiempo durante el cual se generan pulsos de reloj.

### C. Escritura que modifica el canal

Para analizar la modificación del canal de radiofrecuencia, se realizó una escritura en el registro RF_CH del nRF24L01+. La operación se generó al modificar el canal mediante el pulsador correspondiente y confirmar la selección.

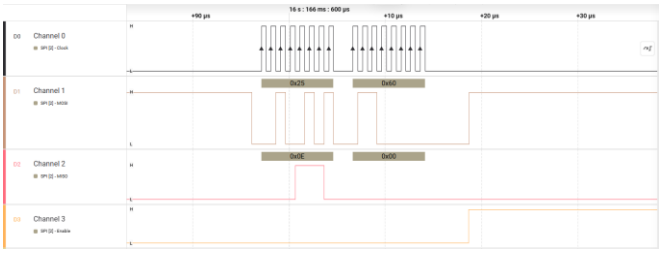



**Figure 4.Escritura del registro RF_CH al cambiar el canal.**

En la captura se identifican los bytes 0x25 y 0x60. El primero es el comando W_REGISTER combinado con la dirección del registro RF_CH: 0x20 OR 0x05 = 0x25. El segundo byte, 0x60, equivale a 96 en decimal y configura el canal 96. La frecuencia correspondiente se calcula con la ecuación (1):

$$f = 2400\,MHz + 96 = 2496\,MHz$$

Por MISO se recibe 0x0E durante el byte de comando y 0x00 durante el byte de datos. En una escritura, este segundo byte no representa una lectura del contenido de RF_CH. CSN pasa a nivel bajo antes de los pulsos de SCK y vuelve a nivel alto al finalizar la transacción.

| Medida | Valor |
|---|---|
| Duración de un byte | 7.583 µs |
| Inicio byte 1 a inicio byte 2 | 9.5 µs |
| Pausa entre bytes | 2 µs |
| Reloj activo (2 bytes) | 17.0 µs |
| CSN en bajo | 75,833 µs |

**Tabla 3. Tiempos medidos en la escritura del registro**

Para dos bytes y f_SCK = 1 MHz, el tiempo ideal calculado mediante la ecuación (4) es:

$$t_{teo} = \frac{8 \times 2}{1\,MHz} = 16\mu s$$

Con un tiempo de reloj activo medido de 17 µs, el error relativo respecto al valor teórico es:

$$E\% = \frac{|17 - 16|}{16} \times 100 = 6.25\ \%$$

La diferencia de 1 µs entre el tiempo ideal (16 µs) y el tiempo de reloj activo medido (17 µs) equivale a un error relativo del 6,25 %. En esta captura, CSN permanece en bajo durante 75,833 µs, es decir, unas 4,46 veces el tiempo de reloj activo (75,833/17 = 4,46). La diferencia de 58,833 µs corresponde al tiempo adicional de la transacción, asociado al control de CSN y a la ejecución del firmware.

### D. Transferencia de un paquete hacia el transceptor

Para analizar la transferencia de información hacia el nRF24L01+, se envió un mensaje desde el nodo transmisor y se localizó la transacción correspondiente al comando W_TX_PAYLOAD (0xA0), utilizado para cargar el paquete que posteriormente podrá transmitirse por radio.

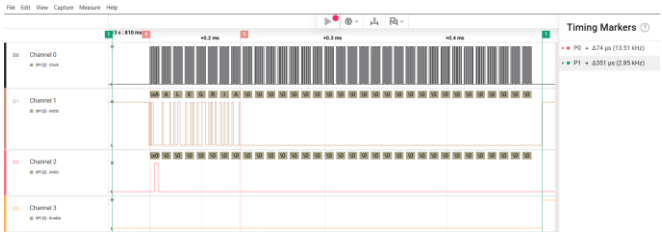



**Figure 5. Vista general**

Como mensaje de prueba se utilizó la palabra ALEGRIA, formada por siete caracteres. En la tabla se presenta el código hexadecimal correspondiente a cada carácter:

| **Letra** | **A** | **L** | **E** | **G** | **R** | **I** | **A** |
|---|---|---|---|---|---|---|---|
| Hex | 0x41 | 0x4C | 0x45 | 0x47 | 0x52 | 0x49 | 0x41 |

**Tabla 4 . Códigos ASCII mensaje ALEGRIA.**

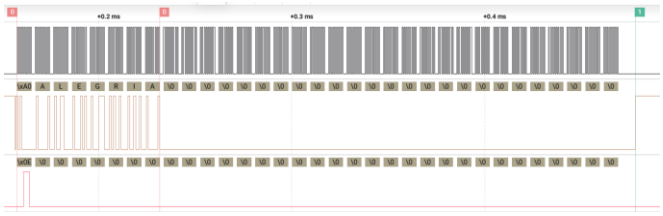


**Figure 6. Envío de Mensaje (Alegría)**

El tiempo de reloj activo desde el comando 0xA0 hasta el último carácter del mensaje fue de 74,0 µs. La transferencia analizada contiene ocho bytes: el byte de comando y los siete caracteres de ALEGRIA. Por tanto, el tiempo ideal de reloj, para n = 8 y f_SCK = 1 MHz, se obtiene con la ecuación (2):

$$t_{teo} = \frac{8 \times 8}{1\ MHz} = 64\ \mu s$$

Con el tiempo de reloj activo medido de 74,0 µs, el error relativo respecto al tiempo teórico se calcula mediante la ecuación (5):

$$E\% = \frac{|74 - 64|}{64} \times 100 = 15.6\ \%$$

La diferencia entre los 74,0 µs medidos y los 64 µs ideales se relaciona con los intervalos entre bytes observados en la captura. Si los bytes comienzan cada 9,5 µs, los ocho bytes ocupan aproximadamente 9,5 × 7 + 7,583 = 74,083 µs. Después de ALEGRIA, la transacción continúa con bytes de relleno 0x00; estos no se incluyen en la medición de los ocho bytes analizados.




**Figure 7. Vista del CSn**

Después de ALEGRIA, la captura muestra 25 bytes de relleno 0x00, de modo que la transacción completa tiene 33 bytes: el comando y 32 bytes de datos, que es la longitud máxima del payload del módulo. Con los bytes iniciándose cada 9,5 µs, el reloj permanece activo durante aproximadamente 9,5 × 32 + 7,583 = 311,6 µs. CSN permanece en bajo durante 351 µs, por lo que el tiempo adicional atribuible al firmware es de unos 351 − 311,6 = 39 µs.

## VI. ANÁLISIS DE RESULTADOS

### A. Comparación de las mediciones experimentales

A continuación se resumen los resultados obtenidos durante las pruebas de lectura de registros, escritura del canal de radiofrecuencia y carga de un paquete de datos en el transceptor nRF24L01+.

| Operación | Datos identificados | Bytes | Tiempo teórico | Tiempo medido | Error relativo |
|---|---|---|---|---|---|
| Lectura de registros | R_REGISTER (0x03+ 0xFF) | 2 | 16 µs | 17 µs | 6,25% |
| Escritura del canal | 0x25, 0x60 | 2 | 16 µs | 17 µs | 6,25% |
| Carga del paquete | 0xA0 +ALEGRIA | 8 | 64 µs | 74 µs | 15,6 % |

**Tabla 5. Comparación entre los tiempos teóricos y medidos en las tres operaciones SPI.**

Las diferencias entre los tiempos teóricos y experimentales se explican principalmente por la temporización de la comunicación SPI. En la lectura y escritura del canal, la

diferencia fue pequeña de 16 a 17 µs, mientras que en la carga de «ALEGRIA» el intervalo aumentó de 64 a 74 µs. Esto ocurre porque el cálculo teórico considera una transferencia continua a 1 MHz, mientras que la ejecución real puede incluir pausas entre bytes debido a la secuencia del programa. Por tanto, el tiempo adicional no implica necesariamente un error en la comunicación.

Además, CSN permaneció en nivel bajo durante más tiempo que la transferencia de datos, ya que delimita la operación completa y no únicamente los pulsos de reloj.

## VII. CONCLUSIONES

1. El análisis de las capturas permitió identificar la estructura de las transacciones SPI del nRF24L01+, reconociendo el comando enviado por MOSI, la respuesta recibida por MISO y el comportamiento de CSN. En la lectura de SETUP_AW se observó el byte STATUS y se recibió 0x03 como contenido del registro, valor que corresponde a un ancho de dirección de cinco bytes.

2.
3. La escritura del canal se realizó con 0x25 (W_REGISTER 0x20 + dirección de RF_CH 0x05) y el dato 0x60 = 96, que corresponde a una portadora de 2496 MHz. Durante una escritura el segundo byte recibido por MISO (0x00) no representa el contenido del registro. Permitiendo asi que no se viera afectado por el rango de operación de wifi

4. La captura de W_TX_PAYLOAD (0xA0) permitió reconocer la carga de los siete caracteres de ALEGRIA, transmitidos después del byte de comando. Esta operación muestra cómo el maestro utiliza SPI para entregar los datos al transceptor antes de la transmisión inalámbrica.

5. Los tiempos de reloj activo fueron de 17 µs para dos bytes y de 74 µs para ocho bytes, frente a tiempos ideales de 16 µs y 64 µs, respectivamente. Los errores relativos son 6,25 % y 15,625 %. Las diferencias concuerdan con los intervalos entre bytes observados en las capturas; por tanto, el tiempo total medido no depende únicamente de la frecuencia nominal de SCK.

6. El tiempo de CSN en bajo fue distinto en cada operación: 101,75 µs en la lectura de registros, 75,833 µs en la escritura de RF_CH y 351 µs en la carga del paquete. En la tercera captura, CSN permanece en bajo 351 µs, aproximadamente 4,74 veces el tiempo de reloj activo (351/74 = 4,74). De ese intervalo, 74 µs corresponden al reloj activo y los 277 µs restantes al tiempo adicional de la transacción. Este tiempo adicional refleja la duración completa de la transacción controlada por el firmware y no debe interpretarse como tiempo de transferencia efectiva de bits.

## VIII. REFERENCIAS

[1] Nordic Semiconductor, nRF24L01+ Single Chip 2.4GHz Transceiver, Product Specification v1.0, 2008. [Online]. Disponible en https://cdn.sparkfun.com/assets/home_ page_ poSts/2/6/7/3/nRF24L01P.pdf.

[2] MicroPython, “class SPI – a Serial Peripheral Interface bus protocol (controller side),” MicroPython Documentation, 2026. [Online]. Disponible en: https://docs.micropython.org/ en/latest/library/machine.SPI.html.

[3] Raspberry Pi Ltd., Raspberry Pi Pico 2 W Datasheet, 2026. [Online]. Disponible en: https://datasheets.raspberrypi.com/ picow/pico-2-w-datasheet.pdf.

[4] Saleae, “SPI Analyzer – User Guide,” Saleae Support, 2026. [Online]. Disponible en: https://support.saleae.com/ protocol-analyzers/ analyzer-user-guides/using-spi.

## IX. ANEXOS

### ANEXO A.

### A1. Código main.py

```
#main.py
from machine import Pin, SPI, I2C
import ssd1306, time
from nrf24 import NRF24

# ---------- Hardware ----------
i2c = I2C(1, sda=Pin(14), scl=Pin(15), freq=400000)
oled = ssd1306.SSD1306_I2C(128, 64, i2c, addr=0x3C)
spi = SPI(0, baudrate=1000000, polarity=0, phase=0,
sck=Pin(18), mosi=Pin(19), miso=Pin(16))
nrf = NRF24(spi, csn=Pin(17, Pin.OUT), ce=Pin(20,
Pin.OUT))

b_var = Pin(13, Pin.IN, Pin.PULL_UP)     # variar entre
opciones
b_sel = Pin(12, Pin.IN, Pin.PULL_UP)     # seleccionar
led_r = Pin(11, Pin.OUT)              # rojo: error / no se
transmitio
led_a = Pin(10, Pin.OUT)              # amarillo: configurando /
esperando
led_v = Pin(9, Pin.OUT)               # verde: enviado / recibido

# ---------- Valores definidos ----------
CANAL_MAX = 125                       # canales 0..125 ->
2400..2525 MHz
POT_TXT = ["-18 dBm", "-12 dBm", "-6 dBm", "0 dBm"]
VELOCIDADES = ["250K", "1M", "2M"]
VEL_TXT = ["250Kbps", "1Mbps", "2Mbps"]
ROLES = ["TX", "RX"]
ENVIOS = ["Unico", "Loop 1 msg", "Loop lista"]     # modo
de envio
```

```
INTERVALOS = [100, 250, 500, 1000, 2000]          # ms
entre envios del loop
IV_TXT = ["100 ms", "250 ms", "500 ms", "1 s", "2 s"]
MENSAJES = [b"ALEGRIA", b"MIGUELITO",
b"ABCDEFGHIJKLMNOPQRSTUVWXYZ",
b"abcdefghijklmnopqrstuvwxyz",
b"ABCDEFGHIJKLMNOPQRSTUVWXYZ123456"]   #
maximo 32 bytes

# configuracion actual ("can" es el numero de canal
directamente: 0..125)
cfg = {"rol": 0, "can": 40, "pot": 3, "vel": 1, "msg": 0, "env":
0, "iv": 3}

# ---------- Utilidades ----------
def leds(r=0, a=0, v=0):
led_r(r)
led_a(a)
led_v(v)

def leer_botones():
"""Devuelve 'var', 'var_largo', 'sel' o None."""
if b_var.value() == 0:
time.sleep_ms(30)
t0 = time.ticks_ms()
while b_var.value() == 0 and time.ticks_diff(time.ticks_ms(),
t0) < 800:
time.sleep_ms(10)
largo = b_var.value() == 0
while b_var.value() == 0:
time.sleep_ms(10)
return "var_largo" if largo else "var"
if b_sel.value() == 0:
time.sleep_ms(30)
while b_sel.value() == 0:
time.sleep_ms(10)
return "sel"
return None

def esperar_boton():
ev = None
while ev is None:
ev = leer_botones()
time.sleep_ms(10)
return ev

def hay_boton():
return b_var.value() == 0 or b_sel.value() == 0

def esperar_soltar():
while hay_boton():
time.sleep_ms(10)
time.sleep_ms(30)

def aplicar():
"""Carga la configuracion elegida en el NRF24L01."""
nrf.ce(0)
nrf.set_channel(cfg["can"])
nrf.set_power(cfg["pot"])
nrf.set_datarate(VELOCIDADES[cfg["vel"]])
if cfg["rol"] == 1:
nrf.start_rx()

# ---------- Pantallas ----------
def verificar_nrf():
if nrf.read_reg(0x03) != 0x03:       # SETUP_AW debe leerse
como 0x03
leds(r=1)
oled.fill(0)
oled.text("ERROR NRF24L01", 0, 20)
oled.text("Revise cableado", 0, 32)
oled.show()
while True:
time.sleep(1)

def inicio():
leds(a=1)
oled.fill(0)
oled.text("Comunicacion", 0, 8)
oled.text("digital", 0, 20)
oled.text("inalambrica", 0, 32)
oled.text("GFSK - UMNG", 0, 48)
oled.show()
time.sleep(2.5)
verificar_nrf()
canal = cfg["can"]
oled.fill(0)
oled.text("CONFIGURACION", 0, 0)
oled.text("Canal: %d" % canal, 0, 14)
oled.text("Freq: %d MHz" % (2400 + canal), 0, 26)
oled.text("Vel: " + VEL_TXT[cfg["vel"]], 0, 38)
oled.text("Pot: " + POT_TXT[cfg["pot"]], 0, 50)
oled.show()
time.sleep(3)

def filas_menu():
return ["Rol: " + ROLES[cfg["rol"]],
"Canal: %d" % cfg["can"],
"Pot: " + POT_TXT[cfg["pot"]],
"Vel: " + VEL_TXT[cfg["vel"]],
"Msg: %d/%d" % (cfg["msg"] + 1, len(MENSAJES)),
"Envio: " + ENVIOS[cfg["env"]],
"Intervalo: " + IV_TXT[cfg["iv"]],
"Iniciar"]

def dibujar_menu(sel):
filas = filas_menu()
ini = max(0, sel - 5)              # ventana de 6 filas con
desplazamiento
oled.fill(0)
oled.text("MENU", 0, 0)
oled.text("%d/%d" % (sel + 1, len(filas)), 96, 0)
for k in range(6):
i = ini + k
if i >= len(filas):
break
```

```
oled.text((">" if i == sel else " ") + filas[i], 0, 9 + 9 * k)
oled.show()

def elegir(titulo, opciones, idx):
"""opciones: lista de (linea1, linea2). GP13 cambia, GP12
acepta."""
while True:
oled.fill(0)
oled.text(titulo, 0, 0)
l1, l2 = opciones[idx]
oled.text(l1, 0, 20)
oled.text(l2, 0, 32)
oled.text("13: cambiar", 0, 46)
oled.text("12: aceptar", 0, 55)
oled.show()
ev = esperar_boton()
if ev in ("var", "var_largo"):
idx = (idx + 1) % len(opciones)
elif ev == "sel":
return idx

def elegir_canal(ch):
"""Canal 0..125. GP13 corto: +1, GP13 largo: +10, GP12:
aceptar."""
while True:
oled.fill(0)
oled.text("CANAL", 0, 0)
oled.text("Canal: %d" % ch, 0, 16)
oled.text("Freq: %d MHz" % (2400 + ch), 0, 28)
oled.text("13:+1 largo:+10", 0, 44)
oled.text("12: aceptar", 0, 55)
oled.show()
ev = esperar_boton()
if ev == "var":
ch = (ch + 1) % (CANAL_MAX + 1)
elif ev == "var_largo":
ch = (ch + 10) % (CANAL_MAX + 1)
elif ev == "sel":
return ch

def menu():
leds(a=1)                    # amarillo: configurando
sel = 0
n = len(filas_menu())
while True:
dibujar_menu(sel)
ev = esperar_boton()
if ev in ("var", "var_largo"):
sel = (sel + 1) % n
elif ev == "sel":
if sel == 0:
cfg["rol"] = elegir("ROL", [("TRANSMISOR", ""),
("RECEPTOR", "")], cfg["rol"])
elif sel == 1:
cfg["can"] = elegir_canal(cfg["can"])
elif sel == 2:
cfg["pot"] = elegir("POTENCIA TX", [(p, "") for p in
POT_TXT], cfg["pot"])
elif sel == 3:
cfg["vel"] = elegir("VELOCIDAD", [(v, "") for v in
VEL_TXT], cfg["vel"])
elif sel == 4:
cfg["msg"] = elegir("MENSAJE",
[(m.decode()[:16], m.decode()[16:32]) for m in MENSAJES],
cfg["msg"])
elif sel == 5:
cfg["env"] = elegir("MODO DE ENVIO", [(e, "") for e in
ENVIOS], cfg["env"])
elif sel == 6:
cfg["iv"] = elegir("INTERVALO LOOP", [(t, "") for t in
IV_TXT], cfg["iv"])
else:
return

def pantalla_run(titulo, texto, extra, estado):
canal = cfg["can"]
oled.fill(0)
oled.text("%s Canal:%d" % (titulo, canal), 0, 0)
oled.text("%dMHz %s" % (2400 + canal,
VEL_TXT[cfg["vel"]]), 0, 9)
oled.text("Pot: " + POT_TXT[cfg["pot"]], 0, 18)
oled.text(texto[:16], 0, 30)
oled.text(texto[16:32], 0, 39)
oled.text(extra[:16], 0, 48)
oled.text(estado[:16], 0, 56)
oled.show()

# ---------- Modo transmisor ----------
def enviar_uno(msg, cont):
"""Envia un mensaje, actualiza contadores y LEDs. Devuelve
True si hubo ACK."""
leds(a=1)
ok = nrf.send(msg)
print("TX:", msg, "OK" if ok else "SIN ACK")
if ok:
cont["ok"] += 1
leds(v=1)
else:
cont["err"] += 1
leds(r=1)
return ok

def loop_tx(cont):
"""Envia en bucle hasta que se presione cualquier boton."""
espera = INTERVALOS[cfg["iv"]]
i = cfg["msg"]
n = 0
while True:
msg = MENSAJES[i] if cfg["env"] == 2 else
MENSAJES[cfg["msg"]]
n += 1
ok = enviar_uno(msg, cont)
pantalla_run("TX", msg.decode(), "OK:%d Err:%d" %
(cont["ok"], cont["err"]),
"Loop #%d" % n)
t0 = time.ticks_ms()
```

```
while time.ticks_diff(time.ticks_ms(), t0) < espera:
if hay_boton():              # cualquier boton detiene el loop
esperar_soltar()
leds(a=1)
return
time.sleep_ms(5)
if cfg["env"] == 2:
i = (i + 1) % len(MENSAJES)

def correr_tx():
aplicar()
cont = {"ok": 0, "err": 0}
estado = "12:Enviar" if cfg["env"] == 0 else "12:Iniciar loop"
leds(a=1)
while True:
msg = MENSAJES[cfg["msg"]]
pantalla_run("TX", msg.decode(), "OK:%d Err:%d" %
(cont["ok"], cont["err"]), estado)
ev = esperar_boton()
if ev == "var":                          # siguiente mensaje
cfg["msg"] = (cfg["msg"] + 1) % len(MENSAJES)
estado = "Msg %d/%d" % (cfg["msg"] + 1, len(MENSAJES))
elif ev == "var_largo":                  # volver al menu
return
elif ev == "sel":
if cfg["env"] == 0:                      # envio unico
pantalla_run("TX", msg.decode(),
"OK:%d Err:%d" % (cont["ok"], cont["err"]), "Enviando...")
ok = enviar_uno(msg, cont)
estado = "Enviado" if ok else "ERROR sin ACK"
pantalla_run("TX", msg.decode(),
"OK:%d Err:%d" % (cont["ok"], cont["err"]), estado)
time.sleep_ms(800)
leds(a=1)
else:                                    # loop
loop_tx(cont)
estado = "Loop detenido"

# ---------- Modo receptor ----------
def correr_rx():
aplicar()
n = 0
texto = ""
estado = "Esperando..."
t_led = 0
leds(a=1)
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
while True:
if nrf.available():
data = nrf.recv().rstrip(b'\x00')
texto = data.decode('utf-8', 'ignore')
n += 1
print("RX:", texto)
leds(v=1)
t_led = time.ticks_add(time.ticks_ms(), 600)
estado = "Recibido"
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
if t_led and time.ticks_diff(time.ticks_ms(), t_led) >= 0:
leds(a=1)
t_led = 0
ev = leer_botones()
if ev == "var_largo":                    # volver al menu
return
if ev == "sel":                          # reiniciar contador
n = 0
texto = ""
estado = "Esperando..."
pantalla_run("RX", texto, "Recibidos: %d" % n, estado)
time.sleep_ms(10)

# ---------- Programa principal ----------
inicio()
while True:
menu()
if cfg["rol"] == 0:
correr_tx()
else:
correr_rx()
```

**A2. Código nrf24**

```
#nrf24.py
import time

class NRF24:
   def __init__(self, spi, csn, ce, payload=32,
addr=b"UMNG1"):
      self.spi, self.csn, self.ce = spi, csn, ce
      self.payload = payload
      self.cfg = 0x0C                # CRC activado, 2 bytes
      self.csn(1)
      self.ce(0)
      time.sleep_ms(100)
      self.write_reg(0x00, self.cfg)     # CONFIG
      self.write_reg(0x01, 0x01)         # auto-ACK en pipe 0
      self.write_reg(0x02, 0x01)         # habilitar pipe 0
      self.write_reg(0x03, 0x03)         # direcciones de 5 bytes
      self.write_reg(0x04, 0x2F)         # 15 reintentos, espera
750 us
      self.write_reg(0x05, 40)           # canal inicial
      self.write_reg(0x06, 0x06)         # 1 Mbps, 0 dBm
      self.write_reg(0x10, addr)         # TX_ADDR
      self.write_reg(0x0A, addr)         # RX_ADDR_P0
      self.write_reg(0x11, payload)      # tamano del payload
      self.cmd(0xE1)                 # flush TX
      self.cmd(0xE2)                 # flush RX
      self.write_reg(0x07, 0x70)         # limpiar banderas

   # ----- SPI basico -----
   def cmd(self, c):
      self.csn(0)
      self.spi.write(bytes([c]))
      self.csn(1)
```

```
    def write_reg(self, reg, val):
        if isinstance(val, int):
            val = bytes([val])
        self.csn(0)
        self.spi.write(bytes([0x20 | reg]) + val)
        self.csn(1)

    def read_reg(self, reg):
        rx = bytearray(2)
        self.csn(0)
        self.spi.write_readinto(bytes([reg, 0xFF]), rx)
        self.csn(1)
        return rx[1]

    def status(self):
        rx = bytearray(1)
        self.csn(0)
        self.spi.write_readinto(b'\xff', rx)
        self.csn(1)
        return rx[0]

    # ----- Configuracion -----
    def set_channel(self, ch):
        # RF_CH admite 0..125  ->  f = 2400 + ch (MHz)  ->
2400 a 2525 MHz
        ch = max(0, min(125, int(ch)))
        self.write_reg(0x05, ch)

    def get_channel(self):
        return self.read_reg(0x05)

    def set_power(self, level):           # 0=-18, 1=-12, 2=-6, 3=0
dBm
        rf = self.read_reg(0x06) & 0xF9
        self.write_reg(0x06, rf | (level << 1))

    def set_datarate(self, vel):          # "250K", "1M" o "2M"
        rf = self.read_reg(0x06) & 0xD7
        rf |= {"250K": 0x20, "1M": 0x00, "2M": 0x08}[vel]
        self.write_reg(0x06, rf)

    # ----- Transmisor -----
    def send(self, data, timeout_ms=300):
        data = bytes(data)[:self.payload]
        data = data + bytes(self.payload - len(data))
        self.ce(0)
        self.write_reg(0x00, self.cfg | 0x02)      # PWR_UP,
modo TX
        time.sleep_ms(2)
        self.cmd(0xE1)
        self.write_reg(0x07, 0x70)
        self.csn(0)
        self.spi.write(b'\xA0' + data)            #
W_TX_PAYLOAD
        self.csn(1)
        self.ce(1)
        time.sleep_us(20)
        self.ce(0)
        t0 = time.ticks_ms()
        while time.ticks_diff(time.ticks_ms(), t0) < timeout_ms:
            st = self.status()
            if st & 0x20:                         # TX_DS: enviado con
ACK
                self.write_reg(0x07, 0x20)
                return True
            if st & 0x10:                         # MAX_RT: sin ACK
                self.write_reg(0x07, 0x10)
                self.cmd(0xE1)
                return False
        self.cmd(0xE1)
        return False

    # ----- Receptor -----
    def start_rx(self):
        self.ce(0)
        self.write_reg(0x00, self.cfg | 0x03)      # PWR_UP,
modo RX
        time.sleep_ms(5)
        self.write_reg(0x07, 0x70)
        self.cmd(0xE2)
        self.ce(1)
        time.sleep_us(150)

    def available(self):
        return not (self.read_reg(0x17) & 0x01)    #
FIFO_STATUS: RX no vacio

    def recv(self):
        rx = bytearray(self.payload + 1)
        self.csn(0)
        self.spi.write_readinto(b'\x61' + bytes(self.payload), rx)
# R_RX_PAYLOAD
        self.csn(1)
        self.write_reg(0x07, 0x40)
        return bytes(rx[1:])
```